

Implementation Self-Assessment Test Report

Key Encapsulation-DKEM-ARM-v1.0 (ICCS 2026-06 electronic submission)

Per the ICCS ARM-architecture Implementation Self-Assessment Guideline (2026-06, CN & EN editions)

This report records, for DKEM (Key Encapsulation (KEM)) on the ARM architecture, the basic information, test environment, functional correctness, performance, resource consumption and transfer/storage sizes, plus a raw-evidence index. Verifiable technical evidence only; no security claim. Object: the will-submit electronic package. Test date: 2026-06-24.

1 Basic Information

- **Algorithm:** DKEM (PKCKEM-380333)
- **Category:** Key Encapsulation (KEM)
- **Implementation version:** v1.0 (ICCS 2026-06 electronic submission)
- **Architecture:** ARM
- **Implementations:** Reference (portable ISO C99) and Optimized (arm64 NEON)
- **Instances & security level:** DKEM-128/-256/-512 — 128/256/512-bit classical security level

Table 1 Parameter sets

Instance	N	Q	K	Security
DKEM-128	256	3329	2	128 -bit
DKEM-256	256	3329	4	256 -bit
DKEM-512	512	7681	4	512 -bit

2 Test Environment

- **Primary hardware:** aarch64 (ARMv8.2-A class), single-core measurement
- **OS:** Ubuntu 24.04 (aarch64)
- **Compiler / build:** GNU gcc 13; bash build.sh
- **Supplementary cross-check:** Apple Silicon (macOS 15, Apple clang) functional KAT cross-check
- **Additional embedded:** Cortex-M4 (QEMU, arm-none-eabi-gcc 14.2.1) — 32-bit ARMv7E-M, outside this (v8.2-A) guideline's scope; additional evidence only
- **Memory footprint:** process peak resident memory $\approx$ 1 776 KB (see §6); static image size in

- **Measurement window (UTC):** 2026-06-23T21:01:00Z → 2026-06-23T21:01:57Z

3 Versions and Compile Configuration (Registration)

3.1 Reference (ISO C99, no SIMD)

```
gcc -std=c99 -Wpedantic -Wall -Wextra -O2
    -fcommon -DDKE_FORCE_SCALAR -DDKE_MODE=<128|256|512> -DDKE_HASH=0 -DDKE_RANDOM=0
```

3.2 Optimized (arm64 NEON)

```
gcc/clang -O3 -march=armv8.2-a -flto -fomit-frame-pointer
    -std=c99 -Wpedantic -Wall -Wextra -fcommon -DDKE_USE_AARCH64 -DDKE_USE_AARCH64_NATIVE
    -DDKE_HASH=0 -DDKE_RANDOM=0
```

Additional Cortex-M4: `arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpv4-sp-d16 -O2 -ffunction-sections -fdata-sections -std=gnu99 -nostdlib -Wl,--gc-sections`.

- **Target ISA:** ARMv8.2-A + NEON (Advanced SIMD; SVE and SHA3/crypto extensions not used)
- **RNG:** deterministic DRNG from API_PKC (DKE_RANDOM=0); KAT seeds from DRNG; reproducible.
- **Hash / XOF:** SM3 only via the official auxfunc (sm3hash/pseudoXOF/pseudohash), DKE_HASH=0; no alternative hash built.
- **Third-party:** the ARM64 NEON NTT / polynomial assembly is adapted from the mlkem-native project (Arm Limited, Hanno Becker, Amin Abdulrahman, Matthias Kannwischer et al.; Apache-2.0 OR ISC OR MIT) - external third-party, outside the B.3 ownership scope; the Cortex-M4 Plantard NTT assembly is by proposal-team member Junhao Huang (ePrint 2022/956, Apache-2.0), within B.3. The portable Reference implementation is plain ISO C99 with no third-party code. See Implementations/LICENSES/.
- **Protected files:** drng.c/.h, auxfunc.c/.h, KAT_*.c byte-for-byte unchanged.

4 Functional Test (KAT)

arm64: Reference + AArch64_NEON, 3 instances. Ubuntu gcc 6/6 PASS; macOS clang 6/6 PASS (KAT byte-identical to Test_Vectors, Ref ≡ NEON). Additional Cortex-M4 self-test 3/3 PASS.

5 Performance Test (average time + average cycles)

Averages (DKE_RANDOM=0, seeds from the deterministic DRNG). N=300 for KEM, N=128 for KEX (both ≥ 100 , per guideline §3.3). Each iteration is timed with

clock_gettime(CLOCK_MONOTONIC); single core throughout.

Note: this arm64 platform has no stable user-space cycle counter (PMU restricted), so average cycles are NA; average wall-clock microseconds (μ s) are authoritative.

Table 2 Average time (μ s, keygen / encaps / decaps)

Instance	Reference	NEON
DKEM-128	56.70 / 64.69 / 72.59	39.05 / 42.14 / 43.32
DKEM-256	154.43 / 159.47 / 169.17	134.25 / 138.08 / 139.44
DKEM-512	542.38 / 564.49 / 586.17	501.63 / 512.78 / 517.89

Table 3 Cortex-M4 (cycles, additional evidence)

Instance	keygen	encaps	decaps
DKEM-128	44 789	54 230	65 453
DKEM-256	161 028	160 087	176 719
DKEM-512	590 469	628 085	707 952

The Optimized speedup is concentrated in the NTT / polynomial arithmetic (hand assembly). The hash is SM3 via the official auxfunc (scalar C, a compliance requirement), so the end-to-end speedup is moderate.

6 Resource Consumption (static image + peak memory)

Static image size from GNU size (text+data+bss, bytes) — storage footprint of code + constants + static data; peak resident memory from getrusage(RUSAGE_SELF).ru_maxrss (KB) — runtime memory footprint. Single core, single process.

Table Static image and peak memory

Instance	Ref static	NEON static	Peak RSS
DKEM-128	NaN B	NaN B	1 776 KB
DKEM-256	NaN B	NaN B	1 776 KB
DKEM-512	NaN B	NaN B	1 776 KB

Note: the Optimized image is markedly larger than Reference due to inlined hand assembly and -flto; peak RSS is essentially flat across instances ($\approx$ 1.8 MB), dominated by the process baseline and on-stack key material.

7 Transfer and Storage Sizes (bytes)

Table DKEM key / ciphertext / shared-secret sizes

Instance	Public key	Secret key	Ciphertext	Shared secret
DKEM-128	800	1 600	800	32
DKEM-256	1 568	3 136	1 600	32
DKEM-512	3 392	6 784	3 136	64

8 Conclusion

All three DKEM instances are functionally correct on the ARM architecture (KAT byte-identical to `Test_Vectors`, `Ref` $\equiv$ `Optimized`) and build reproducibly from the submitted automated scripts. Average time, static image and peak memory, target ISA, compile parameters, RNG method and hash backend are all registered above.

9 Raw Evidence Index

The following raw data and scripts ship with the self-assessment materials package and allow independent reproduction of every result in this report:

- **3-Assessment-Code-and-Data/Raw-Evidence/assess_247.txt** — arm64 Ubuntu gcc raw performance/memory output (with timestamps)
- **3-Assessment-Code-and-Data/Raw-Evidence/assess_143.txt** — arm64 macOS clang functional KAT cross-check output
- **3-Assessment-Code-and-Data/run_assess.sh** — unified performance/memory/KAT harness
- **2-Algorithm-Source-Code/Implementations/.../build.sh** — per-instance deterministic build scripts (source of functional KAT)
- **3-Assessment-Code-and-Data/Test_Vectors/** — reference vectors (byte-for-byte KAT comparison targets)

All timestamps are UTC; measurement host, compiler and window are in §2.